Antmicro

Antmicro grvl

2026-06-11

CONTENTS

CHAPTER

ONE

WHAT IS GRVL?

Grvl (pronounced gravel) is Antmicro's open source library for creating rich, modern, animated graphical user interfaces, developed for use on constrained devices like MCUs.

Grvl is designed to be portable, currently supporting standalone Simple DirectMedia Layer (SDL) based applications and Zephyr RTOS.

With its collection of built-in widgets, XML-based config and support for reconfiguration in runtime, grvl was created with ease of use for both designers and developers in mind.

CHAPTER

TWO

QUICKSTART

2.1 Linking (CMAKE)

2.1.1 Standalone

Pull grvl and link it in your project:

```
add_subdirectory(GRVL_SOURCE_DIR)
target_link_libraries(app PRIVATE grvl)
```

2.1.2 Zephyr

```
set(GRVL_ZEPHYR ON)
add_subdirectory(GRVL_SOURCE_DIR)
target_link_libraries(app PRIVATE grvl)
```

2.2 Minimal example

```
#include <grvl.h>
#include <Manager.h>
...
int main(){
    ...
    // initialize callbacks
    gui_callbacks_t callbacks;
    callbacks.dma_operation = dma_operation;
    callbacks.dma_operation_clt = dma_operation_clt;
    callbacks.dma_fill = dma_fill;
    callbacks.wait_for_vsync = wait_for_vsync;
    callbacks.flipping_completed = flipping_completed;
    callbacks.set_layer_pointer = set_layer_poiner;
    callbacks.gui_printf = gui_printf;
    callbacks.get_timestamp = get_timestamp;
    grvl::Init(&callbacks);

    // initalize manager
    Manager::Initialize(WIDTH, HEIGHT, BPP, IS_FLIPPED);
```

(continues on next page)

(continued from previous page)

```
    // fill the gui media
    Manager::GetInstance()
        .AddFontToFontContainer(FONT_NAME_USED_IN_XML, new GrvlBakedFont(FONT_
↪PATH))
        .AddImageContentToContainer(IMAGE_NAME_USED_IN_XML, new␣
↪ImageContent(IMAGE_PATH))
        .AddCallbackToContainer(CALLBACK_NAME_USED_IN_XML,␣
↪(grvl::Event::CallbackPointer) callback_function)
        .BuildFromXML(XML_PATH)
        .InitializationFinished();

    while(running){
        // update gui
        Manager::GetInstance().MainLoopIteration();
        ...
        platform_specific_wait(); // e.g. SDL_WaitEventTimeout or k_msleep
        ...

        // update specific components
        ProgressBar* progressBar = dynamic_cast<ProgressBar*>(displayManager->
↪GetActiveScreen()->GetElement("progress_bar"));
        progressBar->SetProgressValue(counter % 100);
        ...
        // handle events
        Manager::GetInstance().ProcessTouchPoint(state, x, y);
    }
}
```

For more in-detail examples see *the Examples chapter*.

CHAPTER

THREE

EXAMPLES

3.1 Animating values

First, grab a component by ID:

```
ProgressBar* progressBar = dynamic_cast<ProgressBar*>(displayManager->
↪GetActiveScreen()->GetElement("progress_bar"));
```

Then execute a setter in the main loop:

```
progressBar->SetProgressValue((SDL_GetTicks() / 100) % 100);
```

3.2 Callbacks

There are two ways to add callbacks to GUI components. You can write them in either C++ or JavaScript.

3.2.1 C++

Create the callbacks:

```
void ButtonCallback(Button *Sender, const Event::ArgVector& Args) {
    printf("Button clicked! (%s)\n", Sender->GetID());
}

void SwitchCallback(SwitchButton *Sender, const Event::ArgVector& Args) {
    switch (Sender->GetSwitchState()) {
        case true:
            printf("Switch is ON! (%s)\n", Sender->GetID());
            break;
        default:
            printf("Switch is OFF! (%s)\n", Sender->GetID());
    }
}
```

Add them to the callbacks container:

```
displayManager->AddCallbackToContainer("ButtonCallback",␣
↪(grvl::Event::CallbackPointer)ButtonCallback);
displayManager->AddCallbackToContainer("SwitchCallback",␣
```

(continues on next page)

(continued from previous page)

```
↪(grvl::Event::CallbackPointer)SwitchCallback);
                                        /*^^^^ choose a callback name */
```

3.2.2 JavaScript

Create a file with JavaScript callbacks, e.g.:

```
// callbacks.js

function ButtonCallback(caller) {
    const buttonId = caller.name
    Print("Button clicked! (" + buttonId + ")")
}

function SwitchCallback(caller) {
    const callerName = caller.name
    if (caller.switchState) {
        Print("Switch is ON! (" + callerName + ")")
    } else {
        Print("Switch is OFF! (" + callerName + ")")
    }
}
```

Then include it in your application's XML layout:

```
<script src="callbacks.js"></script>
```

You can specify a non-default working directory for JavaScript files with:

```
JSEngine::SetSourceCodeWorkingDirectory("Scripts/JavaScript/");
```

See *JavaScript documentation* for further reference.

3.2.3 Binding callbacks

Bind callbacks to GUI components in XML by referencing the callback name:

```
<Button id="test_button" x="0" y="0" width="100" height="100" onClick=
↪"ButtonCallback" text="TEST" />
<SwitchButton id="test_switch" x="500" y="500" width="150" height="100"␣
↪onSwitchON="SwitchCallback" onSwitchOFF="SwitchCallback" />
```

3.3 Popups

Adding an element in XML with a callback to show a popup:

```
<Button id="btn" x="0" y="0" width="150" height="100" onClick="ShowPopup('example_
↪popup')" text="Show Popup" />
```

Adding a popup in XML (with an element closing it on callback):

```
<Popup id="example_popup" x="0" y="0" width="300" height="200">
    <button id="close_popup_btn" x="100" y="10" width="100" height="50" onClick=
↪"ClosePopup" text="Close" />
    <label id="popup_label" font="roboto-medium" x="0" y="100" width="300" height=
↪"50" text="This is a Popup." />
</Popup>
```

3.4 Adding images and fonts

Grvl support two font types **Grvl Baked Fonts** (.gbf) and **True Type Fonts** (.ttf). Both formats have their valid uses. You can create a grvl baked font using the provided `gbf` CLI utility, that converts TTFs to GBFs. The baking process allows you to select which codepoints should be included, refer to `gbf --help` for more information.

```
# Build GBF utility application
cmake -B build
cmake --build build --target gbf

# Bake characters into GBF
./build/gbf --ttf ./romfs/fonts/MyFont.ttf --size 18 --gbf ./romfs/fonts/MyFont.
↪gbf --range ascii,0x100-0x200
```

If you want to use TTF fonts in a memory contained enviroment it may be beneficial to create a smaller TTF fonts with some, unused, glyphs removed. You can use the open source `pyftsubset` utility for that. Unused glyphs can also be deleted from a font in Font Forge graphically.

```
# Create a ASCII-only version of the font
pyftsubset ./romfs/fonts/MyFont.ttf --unicodes=U+0020-007E --output-file=./romfs/
↪fonts/MyFont-ascii.ttf
```

Loading both font types is shown below.

```
displayManager->AddFontToFontContainer("my_font_gbf", new GrvlBakedFont(path_to_
↪font));
displayManager->AddImageContentToContainer("my_image", new ImageContent(path_to_
↪image));

// using True Type Fonts
auto data = std::make_shared<grvl::TrueTypeData>(path_to_font);
displayManager->AddFontToFontContainer("my_font_ttf", new TrueTypeFont(data, 18));
```

3.5 Default fonts

When a font is not specified for an element (or the font is not found) grvl will try using the *normal* font - and if that is not present - the *default* font. You may see this in the logs:

```
[WARNING] Font "normal" doesn't exist. Using default font!
[ERROR] Default font doesn't exist.
[WARNING] Font "normal" doesn't exist. Using default font!
```

(continues on next page)

(continued from previous page)

```
[ERROR] Default font doesn't exist.
[WARNING] Font "normal" doesn't exist. Using default font!
[ERROR] Default font doesn't exist.
```

To fix those errors you just need to provide a *normal* (and/or *default*) font, like so:

```
manager.AddFontToFontContainer("normal", new grvl::TrueTypeFont( /* ... */ ));
manager.AddFontToFontContainer("default", new grvl::TrueTypeFont( /* ... */ ));
```

CHAPTER

FOUR

GENERAL API REFERENCE

Warning

doxygenclass: Cannot find class “grvl::Manager” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::ImageContent” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

CHAPTER

FIVE

XML REFERENCE

5.1 XML validity

A schema is available in grvl along with an example that it validates. To check the validity, use an external tool, for example `xmllint`.

```
$ xmllint --schema <path_to_schema.xsd> <path_to_my_xml_grvl_file.xml>
```

5.2 How to construct a grvl XML document?

First define the root of the document.

```
<doc>
    <!-- screens/special components -->
</doc>
```

Then include the screens/special components in the document.

5.3 Generic component/container attributes

Each component has the following attributes:

- id
- x (absolute, in pixels)
- y (absolute, in pixels)
- width
- height
- visible (true/false)
- foregroundColor
- activeForegroundColor
- backgroundColor
- activeBackgroundColor
- borderColor
- activeBorderColor

- borderType (one of: none, box, top, right, bottom, left)
- borderArcRadius
- onClick (callback)
- onPress (callback)
- onRelease (callback)

Containers can also be specified as `selection`, which makes it a single-choice container, meaning that only one child component can be active and will remain active until another component from that container is activated.

5.4 Generic screen attributes

Each screen is a container, so it has all of the attributes from the *previous section* as well as:

- onSlideToLeft (callback)
- onSlideToRight (callback)
- onLongPress (callback)
- onLongPressRepeat(callback)
- collection (string)
- globalPanelVisible (true/false)

It can also contain `<key>` that can have the following attributes:

- id
- onPress
- onLongPress (callback)
- onLongPressRepeat(callback)
- onRelease

5.5 Special attributes

5.5.1 Colors

Uses the `#aarrggbb` format color.

5.5.2 Alignment

One of: `Center`, `Left`, `Right`.

5.5.3 Border type

One of: `none`, `box`, `top`, `right`, `bottom`, `left`.

5.6 List of components/containers

5.6.1 Label

Attributes

- text
- font
- textColor
- alignment

Example

```
<label id="popup_label" font="roboto-medium" x="0" y="100" width="300" height="50
↪" text="This is a label." />
```

5.6.2 Button

Attributes

- text
- font
- image
- textColor
- activeTextColor
- icoChar
- icoFont
- icoColor
- onLongPress
- onLongPressRepeat

Example

```
<button id="close_popup_btn" x="100" y="10" width="100" height="50"␣
↪backgroundColor="#ffffffff" textColor="#ff0000ff" activeTextColor="#ffff00ff"␣
↪onClick="ClosePopup" text="Close" font="roboto-medium" />
```

5.6.3 Clock

Attributes

- font
- alignment
- onRelease (callback)
- foregroundColor

- format (strftime syntax)

Example

```
<clock id="test_clock" x="150" y="10" width="100" height="100" font="roboto-medium
↪" foregroundColor="#fffff000" format="%a %b %d %l:%M" />
```

5.6.4 Slider

Attributes

- frameColor
- selectedFrameColor
- barColor
- activeBarColor
- activeScrollColor
- maxValue
- minValue
- keepBoundaries
- type - Continuous (default), Discrete (divided into steps - see division), List (list indices)
- division
- font
- image
- onValueChange

Example

```
<slider id="vertical_slider" x="300" y="10" width="20" height="100" scrollColor="
↪#ffff0000" activeBarColor="#ff00ff00" />
```

5.6.5 ListItem

Attributes

- text
- font
- description
- descriptionFont
- ActiveTextColor
- descriptionColor
- activeDescriptionColor
- image

- additionalImage
- roundingImage
- onLongPress
- onLongPressRepeat
- type (one of: StdListField, LeftArrowField, RightArrowField, EmptyField, Dots, DoubleImageField, AlarmField)

Example

```
<ListItem backgroundColor="#ffff0000" id="item_1" height="50" type="StdListField"␣
↪text="first" textColor="#ffffffff" font="roboto-medium" />
```

5.6.6 ProgressBar

Attributes

- progressBarColor

Example

```
<ProgressBar id="progress_bar" x="90" y="60" width="50" height="10"␣
↪progressBarColor="#ffffff00" />
```

5.6.7 CircleProgressBar

Attributes

- progressBarColor
- startColor
- endColor
- startAngle
- endAngle
- radius
- thickness
- staticGradient

Example

```
<CircleProgressBar id="circle_progress_bar" x="175" y="120" width="50" height="50
↪" radius="20" thickness="5" startColor="#fffcba03" endColor="#ffa903fc" />
```

5.6.8 SwitchButton

Attributes

Derives all attributes from `Button` with the addition of:

- onSwitchON - callback executed when switching to active state
- onSwitchOFF - callback executed when switching to inactive state
- stateIndicatorWidth/stateIndicatorHeight - size dimensions of state indicator
- stateIndicatorArcRadius - arc radius of state indicator

Example

```
<SwitchButton id="test_switch" x="0" y="60" width="80" height="50"␣
↪stateIndicatorArcRadius="5" stateIndicatorWidth="20" stateIndicatorHeight="20"␣
↪foregroundColor="#fffcba03" backgroundColor="#ffffffff" font="roboto"␣
↪onSwitchON="SwitchCallback" onSwitchOFF="SwitchCallback" />
```

5.6.9 Checkbox

Attributes

Derives all attributes from *SwitchButton*, but renders its state indicator from width and height size dimensions, instead of state indicator parameters.

Example

```
<Checkbox id="active" x="0" y="0" width="18" height="18" backgroundColor="
↪#FFAAAAAA" activeBackgroundColor="#FFFFFFFF" onClick="NotifyAboutStateChange" />
```

5.6.10 Image

Attributes

- contentId

Example

```
<image contentId="minus" x="253" y="131" />
```

5.6.11 GridRow

Example

```
<GridRow id="row_1" backgroundColor="#ffff0000">
        <button id="row_1_btn_1" text="1" font="roboto-medium" width="50" height=
↪"50" backgroundColor="#ff00ffff" />
        <button id="row_1_btn_2" text="2" font="roboto-medium" width="50" height=
↪"50" backgroundColor="#ffff00ff" />
</GridRow>
```

5.6.12 Division

Component designed to be used as a container for other components, with the purpose of making XML layout clean and well-organized. It has a similar purpose to HTML's `div`.

Example

```
<Division x="0" y="0" width="128" height="32" backgroundColor="#FF0E0F10" >
    <Button id="first" text="First" x="0" y="0" width="32" height="32" font=
↪"roboto" backgroundColor="#FF0E0F10" textColor="#FF9EA2A6" />
    <Button id="second" text="Second" x="42" y="0" width="32" height="32" font=
↪"roboto" backgroundColor="#FF0E0F10" textColor="#FF9EA2A6" />
    <Button id="third" text="Third" x="86" y="0" width="32" height="32" font=
↪"roboto" backgroundColor="#FF0E0F10" textColor="#FF9EA2A6" />
</Division>
```

5.6.13 Separator

Used for horizontal separation of content.

Example

```
<Separator id="separator" x="0" y="20" width="128" foregroundColor="#FF2E2E2E" />
```

5.7 List of screens

5.7.1 GridView

Arranges its chilren elements in a grid layout.

Attributes

- overscrollEnabled
- overscrollHeight
- scrollingEnabled
- overscrollColor
- elementWidth
- elementHeight
- verticalOffset

Example

```
<GridView id="grid" x="0" y="0" width="140" height="140" backgroundColor="
↪#FF0A0A0A" elementWidth="40" elementHeight="40" horizontalOffset="5"␣
↪verticalOffset="5" selection="true" >

    <GridRow id="monthdays1" >
        <button id="0" text="0" font="mona14" backgroundColor="#FF0A0A0A"␣
↪activeBackgroundColor="#FF0059EC" textColor="#FFECEDEE" onClick="SomeCallback" /
↪>
        <button id="1" text="1" font="mona14" backgroundColor="#FF0A0A0A"␣
↪activeBackgroundColor="#FF0059EC" textColor="#FFECEDEE" onClick="SomeCallback" /
↪>
```

(continues on next page)

(continued from previous page)

```
        <button id="2" text="2" font="mona14" backgroundColor="#FF0A0A0A"␣
↪activeBackgroundColor="#FF0059EC" textColor="#FFECEDEE" onClick="SomeCallback" /
↪>
    </GridRow>

    <GridRow id="monthdays2" >
        <button id="3" text="3" font="mona14" backgroundColor="#FF0A0A0A"␣
↪activeBackgroundColor="#FF0059EC" textColor="#FFECEDEE" onClick="SomeCallback" /
↪>
        <button id="4" text="4" font="mona14" backgroundColor="#FF0A0A0A"␣
↪activeBackgroundColor="#FF0059EC" textColor="#FFECEDEE" onClick="SomeCallback" /
↪>
        <button id="5" text="5" font="mona14" backgroundColor="#FF0A0A0A"␣
↪activeBackgroundColor="#FF0059EC" textColor="#FFECEDEE" onClick="SomeCallback" /
↪>
    </GridRow>

    <GridRow id="monthdays3" >
        <button id="6" text="6" font="mona14" backgroundColor="#FF0A0A0A"␣
↪activeBackgroundColor="#FF0059EC" textColor="#FFECEDEE" onClick="SomeCallback" /
↪>
        <button id="7" text="7" font="mona14" backgroundColor="#FF0A0A0A"␣
↪activeBackgroundColor="#FF0059EC" textColor="#FFECEDEE" onClick="SomeCallback" /
↪>
        <button id="8" text="8" font="mona14" backgroundColor="#FF0A0A0A"␣
↪activeBackgroundColor="#FF0059EC" textColor="#FFECEDEE" onClick="SomeCallback" /
↪>
    </GridRow>

</GridView>
```

5.7.2 ListView

Arranges its chilren elements into a list.

Attributes

- overscrollEnabled
- overscrollHeight
- splitLineColor
- overscrollColor
- scrollIndicatorColor

Example

```
<ListView id="list" x="0" y="0" width="200" height="200">

    <ListItem backgroundColor="#ffff0000" id="item1" height="50" type=
```

(continues on next page)

(continued from previous page)

```
↪"StdListField" text="first" textColor="#ffffffff" font="mona12" />
    <ListItem backgroundColor="#ff00ff00" id="item2" height="50" type=
↪"StdListField" text="second" textColor="#ffffffff" font="mona12" />
    <ListItem backgroundColor="#ff0000ff" id="item3" height="50" type=
↪"StdListField" text="third" textColor="#ffffffff" font="mona12" />

</ListView>
```

5.7.3 ScrollPanel

Allows to arrange its children elements inside the container in a custom way, defined by each component's x and y.

Attributes

- overscrollEnabled
- overscrollHeight
- splitLineColor
- overscrollColor
- scrollIndicatorColor

Example

```
<ScrollPanel id="scroll" x="0" y="0" width="100" height="100" overscrollBarColor="
↪#FF0E0F10" >

    <Button id="add" text="Add" font="mona12" x="5" y="5" width="40" height="40"␣
↪textColor="#FFFF575E" onClick="SomeCallback" />
    <Button id="new" text="New" font="mona12" x="55" y="5" width="40" height="40"␣
↪textColor="#FFFF575E" onClick="SomeCallback" />
    <Button id="cancel" text="Cancel" font="mona12" x="5" y="50" width="90"␣
↪height="40" textColor="#FFFF575E" onClick="SomeCallback" />

</ScrollPanel>
```

5.7.4 Other screens

Screens with no special attributes

CustomView

```
<customView id="start" backgroundColor="#FF4287F5">
        <button id="test_button" image="button_image" x="10" y="10" width="100"␣
↪height="100" textColor="#ff00ff00" activeTextColor="#ffff00ff" onClick=
↪"ButtonCallback" text="TEST" font="roboto-medium" />
    <clock id="test_clock" x="150" y="10" width="100" height="100" font="roboto-
↪medium" foregroundColor="#fffff000" format="%H:%M:%S" />
```

(continues on next page)

(continued from previous page)

```
    <slider id="vertical_slider" x="300" y="10" width="20" height="100"␣
↪scrollColor="#ffff0000" activeBarColor="#ff00ff00" />
    <slider id="horizontal_slider" x="340" y="10" width="100" height="20"␣
↪scrollColor="#fffcba03" activeBarColor="#ffa903fc" />
</customView>
```

Header

```
<header height="50" backgroundColor="#ff000000">
    <label id="h_label" text="Header (Label)" font="roboto-medium" alignment=
↪"Center" x="0" y="0" width="800" height="50" />
</header>
```

Popup

```
<popup id="test_popup" backgroundColor="#ff000000" x="250" y="200" width="300"␣
↪height="200">
        <button id="close_popup_btn" x="100" y="10" width="100" height="50"␣
↪backgroundColor="#ffffffff" textColor="#ff0000ff" activeTextColor="#ffff00ff"␣
↪onClick="ClosePopup" text="Close" font="roboto-medium" />
        <label id="popup_label" font="roboto-medium" x="0" y="100" width="300"␣
↪height="50" text="This is a Popup." />
</popup>
```

5.8 List of special components

5.8.1 guiConfig

Attributes

- touchRegionModificator
- dotColor
- dotActiveColor
- dotDistance
- dotRadius
- dotYPos
- debugDot

Example

```
<guiConfig dotColor="#ffffffff"></guiConfig>
```

5.8.2 stylesheet (UNIMPLEMENTED)

The current version of grvl only implements parsing, the stylesheet is not applied.

Example

```
<stylesheet>
    label {
        bgColor: "red"
    }
</stylesheet>
```

5.8.3 keypadMapping

Contains key elements with the following attributes:

- id
- code (int)
- repeat (int)

Example

```
<keypadMapping>
    <key id="space" code="456" repeat="200"/>
    <key id="enter" code="789" repeat="100"/>
</keypadMapping>
```

CHAPTER

SIX

JAVASCRIPT REFERENCE

6.1 JavaScript feature support

The JavaScript engine used in grvl is duktape. For more information about its capabilities, see duktape docs.

6.2 How to use JavaScript with grvl?

First, prepare your JavaScript source file:

```
// callbacks.js

var currentDate = null

function ButtonCallback(caller) {
    const buttonId = caller.name
    Print("Button clicked! (" + buttonId + ")")
}

function InitializeApplication() {
    currentDate = new Date()
}
```

Then, include it in your application's XML layout:

```
<script src="callbacks.js"></script>
```

You can specify a non-default working directory for JavaScript files with:

```
grvl::JSEngine::SetSourceCodeWorkingDirectory("Scripts/JavaScript/");
```

You can later bind JavaScript functions to GUI components by referencing a function name in callback parameters, e.g. `onClick`. The caller component will be passed in as the first argument:

```
<button id="button" x="10" y="140" width="200" height="80" onClick="ButtonCalback
↪" text="I am a button" font="roboto" />
```

JavaScript functions can be also called from any place in code without involving GUI components like this:

```
grvl::JSEngine::MakeJavaScriptFunctionCall("InitializeApplication");
```

6.3 grvl functions available in JavaScript

You can call functions exposed by grvl to control your application from JavaScript. They are divided into two types: globally available functions and GUI component accessors.

6.3.1 Globally available functions

These functions can be called from any place in JavaScript code:

- `GetElementById(componentID)` - returns component with given ID if found
- `Print(message)` - prints given message
- `ShowPopup(popupID)` - shows popup with given ID if available
- `ClosePopup()` - closes currently shown popup
- `SetActiveScreen(screenID)` - sets screen with given ID as active
- `GetTopPanel()` - returns top panel component
- `GetBottomPanel()` - returns bottom panel component
- `GetPrefabById(prefabID)` - returns prefab with given ID if available

6.3.2 Members

These functions can be used to access GUI component parameters or call their member functions.

Properties

The simple parameters of a GUI component can be modified with properties exposed by each component, e.g.:

```
Print(caller.name)
```

will print `caller`'s ID. There are other common properties such as:

- `x`, `y`, `width` and `height` for position and size dimensions respectively
- `foregroundColor` and `backgroundColor` for component's foreground and background color
- `visibility` which determines if a component is visible.

Metadata

Each component's metadata can be accessed by calling `AddMetadata(key, value)` or `GetMetadata(key)`:

```
button.AddMetadata("description", "grvl is very cool")
...
const description = button.GetMetadata("description")
```

Cloning

GUI components can be cloned inside JavaScript code by calling:

```
const clone = caller.Clone()
container.AddElement(clone)
```

CHAPTER

SEVEN

PREFABS

Prefabs are user-defined composites of already existing GUI components and can be used to instantiate complex structures at runtime. By using prefabs you can create new instances of hierarchical GUI components without delving into their internals.

Prefabs can be defined in application's XML layout:

```
<prefab id="event" width="95" height="60" borderType="left" >

    <label id="name" text="team planning" font="mona16" x="0" y="0" width="138"␣
↪height="35" horizontalOffset="8" textColor="#FF47A8FF" alignment="left" />
    <label id="description" font="mona14" x="0" y="25" width="138" height="35"␣
↪horizontalOffset="8" textColor="#FF47A8FF" alignment="left" visibility="false" /
↪>

</prefab>
```

and then later instantiated at runtime with:

```
const prefab = GetPrefabById("event")
const event = prefab.Clone()
events.AddElement(event)
```

CHAPTER

EIGHT

METADATA

Metadata gives an ability to store complex data and information for later use in GUI components. Metadata values can be accessed by calling components' member functions `AddMetadata(key, value)` and `GetMetadata(key)`:

```
button.AddMetadata("description", "grvl is very cool")
...
const description = button.GetMetadata("description")
```

CHAPTER

NINE

ON-SCREEN KEYBOARD

To provide an interactive experience for the user, you can add a keyboard, along with text inputs, to your grvl application. The keyboard can be specified in the application XML layout:

```
<keyboard id="keyboard" backgroundColor="#FF0D0D0E" x="0" y="430" width="1024"␣
↪height="338">

    <KeyboardKey id="g" text="g" font="mona26" secondaryText="a"␣
↪secondaryTextFont="mona14" x="25" y="27" width="75" height="62" backgroundColor=
↪"#FF191A1C" textColor="#FFECEDEE" secondaryTextColor="#FF191A1C" onClick=
↪"AppendFromKeyboardKey" />
    <KeyboardKey id="r" text="r" font="mona26" secondaryText="n"␣
↪secondaryTextFont="mona14" x="115" y="27" width="75" height="62"␣
↪backgroundColor="#FF191A1C" textColor="#FFECEDEE" secondaryTextColor="#FF191A1C
↪" onClick="AppendFromKeyboardKey" />
    <KeyboardKey id="v" text="v" font="mona26" secondaryText="t"␣
↪secondaryTextFont="mona14" x="205" y="27" width="75" height="62"␣
↪backgroundColor="#FF191A1C" textColor="#FFECEDEE" secondaryTextColor="#FF191A1C
↪" onClick="AppendFromKeyboardKey" />
    <KeyboardKey id="l" text="l" font="mona26" secondaryText="m"␣
↪secondaryTextFont="mona14" x="295" y="27" width="75" height="62"␣
↪backgroundColor="#FF191A1C" textColor="#FFECEDEE" secondaryTextColor="#FF191A1C
↪" onClick="AppendFromKeyboardKey" />

</keyboard>
```

You can specify any layout you require by defining individual keys. The keyboard will be shown whenever a text input is used.

```
<TextInput id="name" basicText="Name" font="mona26"
    x="0" y="0" width="426" height="34"
    borderType="box" borderArcRadius="12"
    backgroundColor="#FF191A1C" borderColor="#FF2C2E32" textColor="#FF9EA2A6" />
```

CHAPTER

TEN

WIDGET API REFERENCE

Warning

doxygenclass: Cannot find class “grvl::Label” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::Button” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::Clock” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::Image” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::ProgressBar” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::CircleProgressBar” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::GridView” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::GridRow” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::CustomView” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::ListView” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::ListItem” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::Panel” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::SwitchButton” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml

Warning

doxygenclass: Cannot find class “grvl::Slider” in doxygen xml output for project “grvl” from directory: /home/runner/work/grvl/grvl/build/doxydocs/xml